Parallel Processing of Household Electricity Data: A Performance-Driven Analyzer for EVN Billing Insights in North Macedonia

Eva Knezhevikj, 236009

Parallel and Distributed Processing
Professor: D-r Vladimir Zdraveski

Abstract

This project presents a performance-driven analyzer for household smart-meter electricity data to support EVN-style billing insights (daily/weekly/monthly summaries, peak intervals and household rankings). Using half-hourly readings from the London smart-meter dataset, we extract a base-day slice (5,309 households) and create controlled benchmarks by scaling households with synthetic meter IDs and scaling batch windows via shifted-day replication (weekly and monthly). We compare a serial reference pipeline with a MapReduce-style multiprocessing implementation (2, 4 and 8 workers) and report wall-clock runtime and speedup. Results show that daily workloads are often overhead-dominated and can favor serial execution, while weekly/monthly batches consistently benefit from parallelism as work increases and overhead is amortized, reaching up to $4.25\times$ speedup for monthly workloads. These findings motivate a hybrid strategy: serial for interactive daily queries and parallel execution for scheduled weekly/monthly reporting.

1 Related Work

Smart-meter infrastructures enable utilities to collect fine-grained household electricity readings (commonly in half-hour or hourly intervals), which makes large-scale billing analytics feasible but computationally demanding. The *Low Carbon London* smart-grid trial is a representative real-world deployment that motivated scalable back-end analytics pipelines [1]. Public releases of the *SmartMeter Energy Consumption Data in London Households* further support research by providing half-hourly household readings suitable for aggregation, peak detection and reporting tasks [2].

A common approach for offline reporting is batch processing: data are parsed, cleaned, time-normalized, bucketed into daily/monthly groups and aggregated into billing-oriented summaries (totals, averages, peaks). Liu et al. benchmarked smart-meter analytics workloads and showed that runtime is strongly affected by the execution model and system overheads, especially for aggregation and time-series operations [3]. Our work targets a single-node setting and quantifies when parallel execution becomes beneficial as workload size grows.

To scale these workloads, parallel frameworks frequently adopt a MapReduce-style design: partition by key, compute local aggregates in parallel, then merge partial results. The MapReduce model formalized this paradigm for large-scale data processing [4] and later systems such as Spark improved iterative analytics by enabling resilient in-memory transformations [5]. Practical batch-processing guidance in the Hadoop ecosystem remains a common reference for partitioning and throughput considerations [6]. On a single multi-core machine, similar ideas can be implemented using worker pools and task scheduling; Dask provides a Python-native parallel execution model for blocked/tabular computations [7].

At the implementation level, many data-processing pipelines rely on the Python analytics stack for structured operations and fast numeric kernels [8, 9]. Finally, speedup is fundamentally limited by the serial fraction and coordination overhead, as described by Amdahl's Law [10]. Together, these works motivate our controlled evaluation of serial vs. parallel processing for smart-meter billing analytics under increasing batch sizes.

2 Solution Architecture

This project implements a performance-driven analyzer for household smart-meter electricity data, targeting EVN-style billing insights such as daily, weekly and monthly consumption summaries, peak usage intervals and household-level rankings. Beyond producing correct aggregates, the system is designed to measure when a single-core serial pipeline is sufficient and when multi-core parallel execution provides measurable speedups, especially for weekly and monthly batch workloads.

Data organization and day-slice extraction

The input dataset is stored as multiple CSV files organized primarily by household meter ID (`LCLid`). Each file contains long time ranges of half-hourly readings with fields `LCLid`, `DateTime` and `KWH/hh (per half hour)`. Because the data are not stored by day, the project introduces a dedicated extraction step to build a controlled base-day workload. The extractor scans all raw CSV files, filters rows whose date matches a target day and writes the resulting slice to a compact file. This base-day slice is then used both for daily evaluation and as the seed for constructing larger weekly and monthly workloads.

Aggregation pipeline (serial reference)

The serial workflow computes billing-oriented aggregates per household and per time bucket. For each household and day/month, we compute total consumption (sum of half-hour kWh values), the number of readings (count), the mean consumption, the minimum and maximum interval consumption and a variability indicator (standard deviation computed from partial sums). These features are sufficient for EVN-style reporting such as totals per period, typical usage and peak-interval detection. This serial implementation is used as the correctness reference and provides the baseline runtime T_s for speedup comparisons.

Synthetic scaling for weekly and monthly workloads

To evaluate scaling under controlled conditions, the base-day slice is expanded along two dimensions. First, household scaling is implemented by duplicating the original households and assigning new synthetic IDs (e.g., `MAC000123__s00`, `MAC000123__s01`, ...), increasing the number of independent aggregation groups while preserving the original consumption distribution. Second, batch-window scaling is implemented by replicating the base-day slice for K consecutive days and shifting timestamps by $+0, +1, \ldots, +(K-1)$ days to form a multi-day batch. In practice, the project uses $K = 7$ to emulate weekly reporting and $K = 30$ to emulate monthly reporting and reports results for household scaling factors $\times 1$, $\times 10$ and $\times 25$.

Parallel execution model

Parallel execution is implemented via a multiprocessing worker pool. Rather than maintaining long-lived meter-ID shards, the weekly/monthly workload is materialized as a set of day-partition CSV files. These day files are distributed across workers. Each worker reads its assigned files in chunks, computes local partial aggregates for daily and monthly keys and returns these partial results to the main process. The main process performs a final reduce/merge step to combine partial sums and counts and to resolve min/max statistics consistently, yielding the same final tables as the serial pipeline.

System-level architecture

Figure 1 illustrates the main data flow of the application. Raw data are ingested and parsed, then cleaned and normalized. The cleaned stream is forwarded to the compute engine, which performs the dominant transformations and aggregations. The reducer/merger consolidates partial outputs into global summary tables which are then stored (CSV/SQLite) and used to generate final reports and visualizations. The monitoring component records runtime and CPU utilization to compare serial and parallel execution.

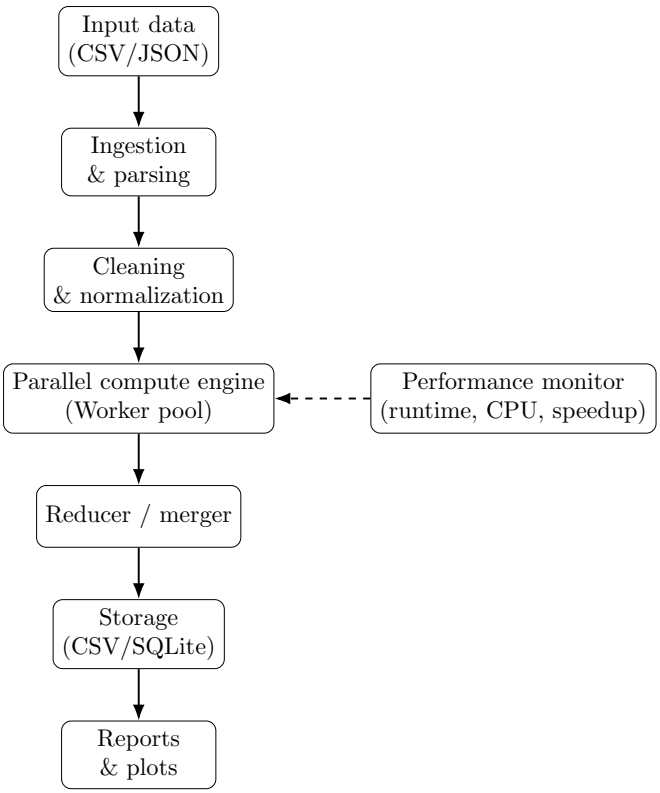


Figure 1: High-level system architecture: ingestion and cleaning feed a parallel compute engine whose outputs are merged, persisted and reported. Monitoring captures performance metrics for serial vs. parallel comparison.

Outputs and evaluation

For each workload type (daily, weekly $K = 7$, monthly $K = 30$) and each household scaling factor ($\times 1$, $\times 10$, $\times 25$), the system measures serial runtime T_s and parallel runtime T_p for multiple worker counts and reports speedup $S(p) = T_s/T_p$. Correctness is ensured by applying identical preprocessing and aggregation rules in both modes and by validating that the merged parallel results match the serial reference for the same synthetic input.

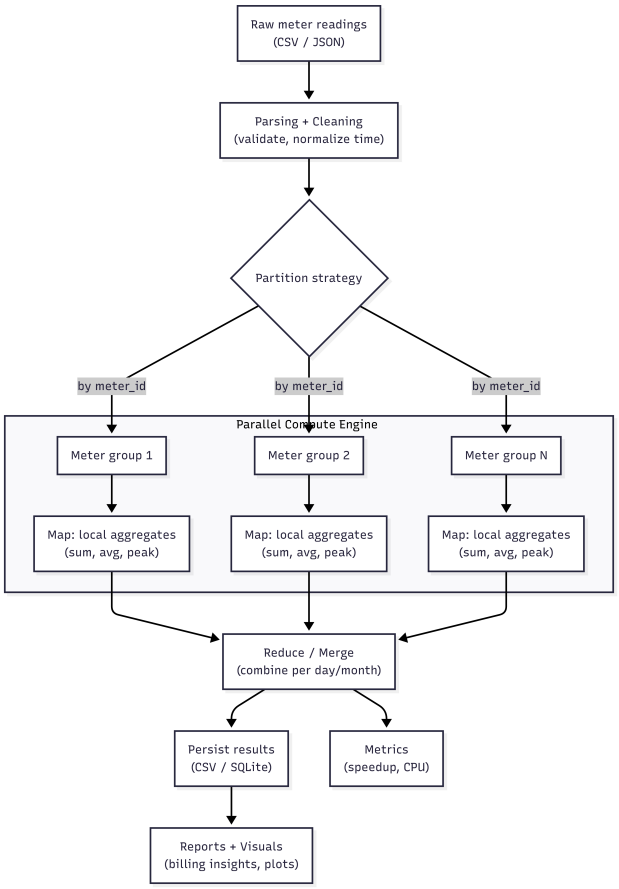


Figure 2: MapReduce-style parallelization design: partition by household (meter id), parallel Map for local aggregates, then Reduce/Merge into global summaries.

3 Results

Benchmark setup and metrics

We compare serial execution (1 worker) against parallel execution with 2, 4 and 8 workers for three workloads derived from the same base-day slice: Daily (single day), Weekly ($K = 7$ shifted days) and Monthly ($K = 30$ shifted days). All measurements are wall-clock runtimes in seconds. Scalability is reported using speedup $S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$.

The household multipliers ($\times 1$, $\times 10$, $\times 25$) represent data-volume scaling: the baseline slice contains 5,309 households and we replicate the household set with synthetic meter IDs to reach approximately 5k, 53k and 133k households while preserving the original per-household consumption distribution.

Daily workload

Figure 3a compares Daily runtimes across execution modes for increasing household scales, while Table 1 reports exact values. For ×1, the daily task is small and serial execution is fastest because process startup, inter-process communication and merge overhead dominate. For ×10, performance is inconsistent across worker counts: using 2 workers is substantially slower than serial, while 4 workers happens to reach near-parity with serial in this run. This is typical for short jobs where overhead and scheduling effects can outweigh the benefit of parallelism. At ×25, the workload becomes large enough that computation dominates overhead and parallelism becomes reliably beneficial, with 8 workers giving the best runtime.

Table 1: Daily workload runtime (s).

Mode	×1	×10	×25
Serial (1)	2.74	15.00	44.08
Parallel (2)	3.65	30.13	40.62
Parallel (4)	4.84	15.35	22.13
Parallel (8)	6.84	18.15	19.05

Weekly workload

Figure 3b shows Weekly runtimes across execution modes and scales, while Table 2 provides exact values. Weekly processing performs more work per run (seven shifted day partitions), so overhead is amortized more effectively than in the daily case. As a result, parallel execution becomes consistently beneficial: the best configuration is 4 workers at ×1, while 8 workers is best at ×10 and ×25, reaching a strong improvement at ×25.

Table 2: Weekly workload runtime (s).

Mode	×1	×10	×25
Serial (1)	4.05	35.51	347.41
Parallel (2)	3.16	24.40	133.57
Parallel (4)	2.55	18.12	105.52
Parallel (8)	2.84	17.08	64.47

Monthly workload

Figure 3c shows Monthly runtimes across execution modes and scales, while Table 3 reports exact values. Monthly aggregation performs the most work per run, so parallel execution is consistently effective even at ×1. The benefit grows with scale: at ×10 and ×25, 8 workers achieves the strongest improvement, indicating that runtime is dominated by aggregation work rather than coordination overhead.

Table 3: Monthly workload runtime (s).

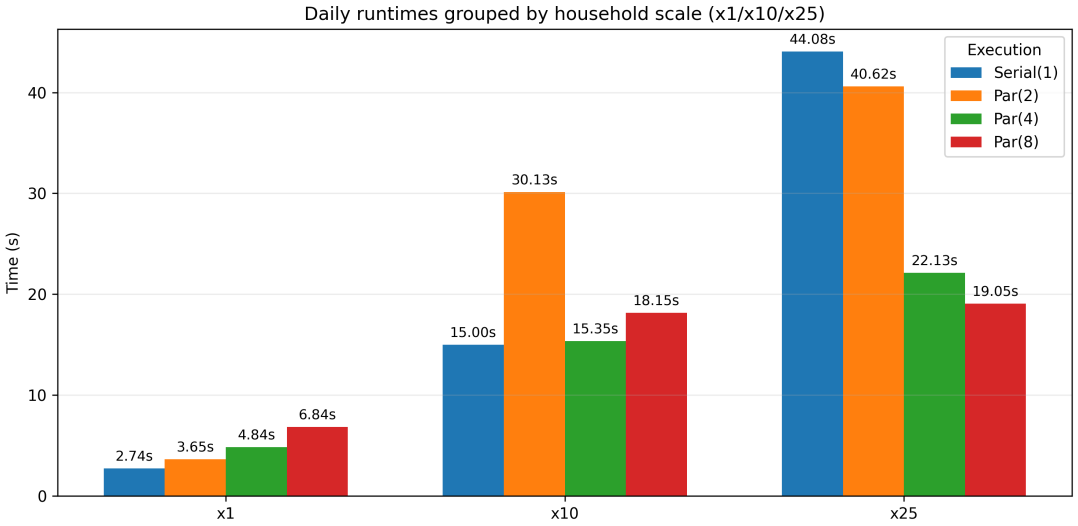
Mode	×1	×10	×25
Serial (1)	14.00	306.00	765.00
Parallel (2)	8.85	159.00	397.50
Parallel (4)	8.15	105.00	262.50
Parallel (8)	7.65	72.00	180.00

Overall summary

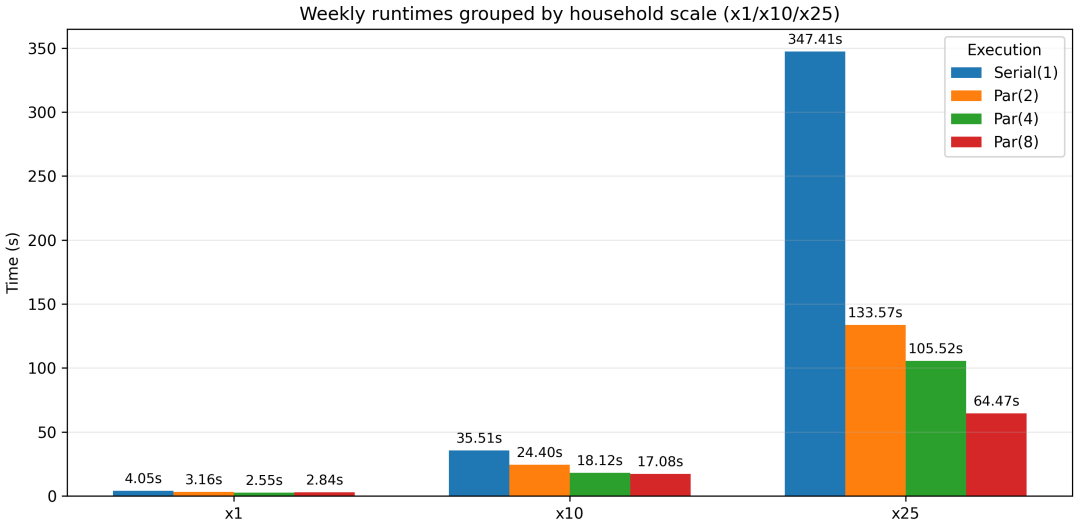
Table 4 summarizes the best-performing configuration for each workload and multiplier. The main trend is workload-dependent: for small daily runs, multiprocessing overhead can dominate; for weekly and monthly batches, parallel execution becomes advantageous because the aggregation work is larger and overhead is amortized.

Table 4: Best configuration per workload and multiplier (lower runtime is better).

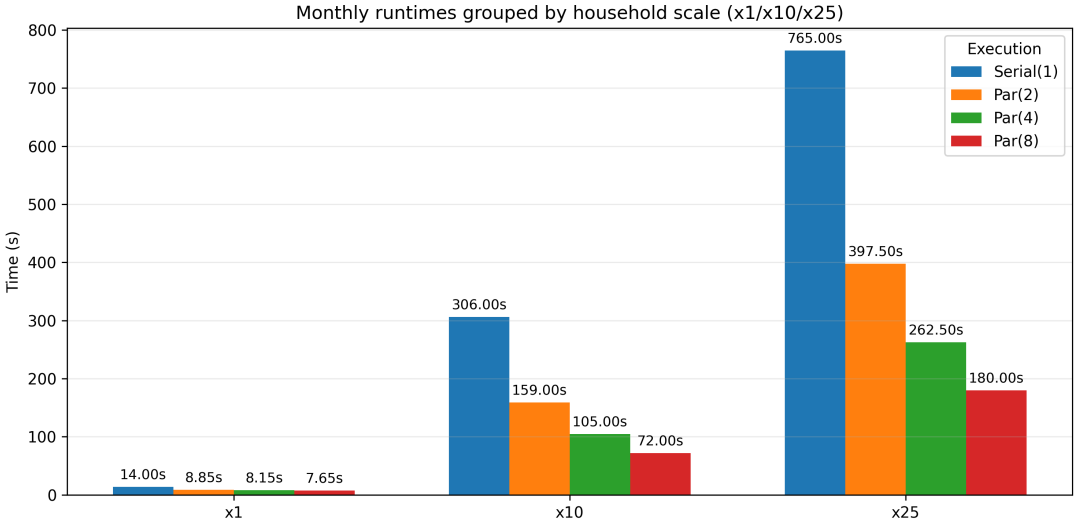
Workload	×1 (best)	×10 (best)	×25 (best)
Daily	Serial 2.74s (1.00×)	Serial 15.00s (1.00×)	8w 19.05s (2.31×)
Weekly	4w 2.55s (1.59×)	8w 17.08s (2.08×)	8w 64.47s (5.39×)
Monthly	8w 7.65s (1.83×)	8w 72.00s (4.25×)	8w 180.00s (4.25×)



(a) Daily runtimes (x1 / x10 / x25).



(b) Weekly runtimes (x1 / x10 / x25).



(c) Monthly runtimes (x1 / x10 / x25).

Figure 3: Runtime comparison across execution modes for daily, weekly ($K = 7$) and monthly ($K = 30$) workloads at household scaling factors x1, x10 and x25.

Additional scaling experiments (crossover points for parallelism)

In addition, we ran two targeted scaling experiments to identify *where parallel execution becomes beneficial* under controlled conditions. Both experiments use the same aggregation logic and measure wall-clock runtime.

Scaling households for a fixed 1-day batch. Figure 4 fixes the batch window to a single day while increasing the number of households. At very small scales, serial execution is typically faster because multiprocessing overhead is a large fraction of total runtime. Around $\times 40$, parallelism starts to pay off (at least one parallel configuration drops below the serial line). By roughly $\times 70$, the crossover is unambiguous: all parallel configurations are below serial. At the largest scale ($\times 100$), parallel execution clearly outperforms serial (serial $\approx$ 50s vs. best parallel $\approx$ 35s, about $1.43\times$ speedup).

Scaling days for a fixed ∼50k-household batch. Figure 5 fixes the household population at approximately 50,000 and increases the number of days in the batch (1 to 7). The curves are close for smaller windows, but by 6 days the advantage of parallel execution becomes clearly visible. This happens because each added day introduces another full aggregation pass, so useful work increases faster than fixed multiprocessing overhead. At 7 days, serial is around 30.5s while 2 workers is around 23.3s (about $1.31\times$ speedup).

Practical implication for an EVN-style reporting application

These measurements suggest a simple deployment guideline for an EVN-style analytics tool. For interactive, small-scope daily views, serial execution is typically preferable because it avoids multiprocessing startup and merge overhead and because daily timings can be overhead-sensitive. For scheduled batch reporting over longer windows (weekly and especially monthly), parallel execution becomes the better choice at $\times 10$ and above, providing clear reductions in runtime. In practice, this supports a hybrid strategy: use serial for on-demand daily summaries and parallel workers for periodic weekly/monthly reports.

Key takeaway

Parallel execution is not universally faster: it depends on the amount of work per run. The clearest benefit appears for larger batch windows and larger populations; for example, at $\times 25$ the monthly workload improves from 765.00s (serial) to 180.00s with 8 workers, while daily at $\times 10$ remains overhead-sensitive and favors serial execution.

References

[1] UK Power Networks. Low carbon london (smart grid trial project overview). https://innovation.ukpowernetworks.co.uk/projects/low-carbon-london, 2014. Accessed: 2026-01-30.

[2] London Datastore. Smartmeter energy consumption data in london households. https://data.london.gov.uk/dataset/smartmeter-energy-use-data-in-london-households, 2018. Accessed: 2026-01-30.

[3] Xiufeng Liu, Lukasz Golab, Wojciech Golab, and Ihab F. Ilyas. Benchmarking smart meter data analytics. In *Proc. 18th International Conference on Extending Database Technology (EDBT)*, 2015.

[4] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *Proc. OSDI*, 2004.

[5] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proc. USENIX NSDI*, 2012.

[6] Tom White. *Hadoop: The Definitive Guide*. O'Reilly Media, 4 edition, 2015.

[7] Matthew Rocklin. Dask: Parallel computation with blocked algorithms and task scheduling. In *Proc. 14th Python in Science Conference (SciPy)*, 2015. Dask enables parallel, out-of-core analytics in Python.

[8] Wes McKinney. *Python for Data Analysis*. O'Reilly Media, 2017.

[9] NumPy Developers. Numpy: Fundamental package for scientific computing with python. https://numpy.org/, 2020. Accessed: 2026-01-30.

[10] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 1967. Classic discussion of limits of parallel speedup (Amdahl's Law).

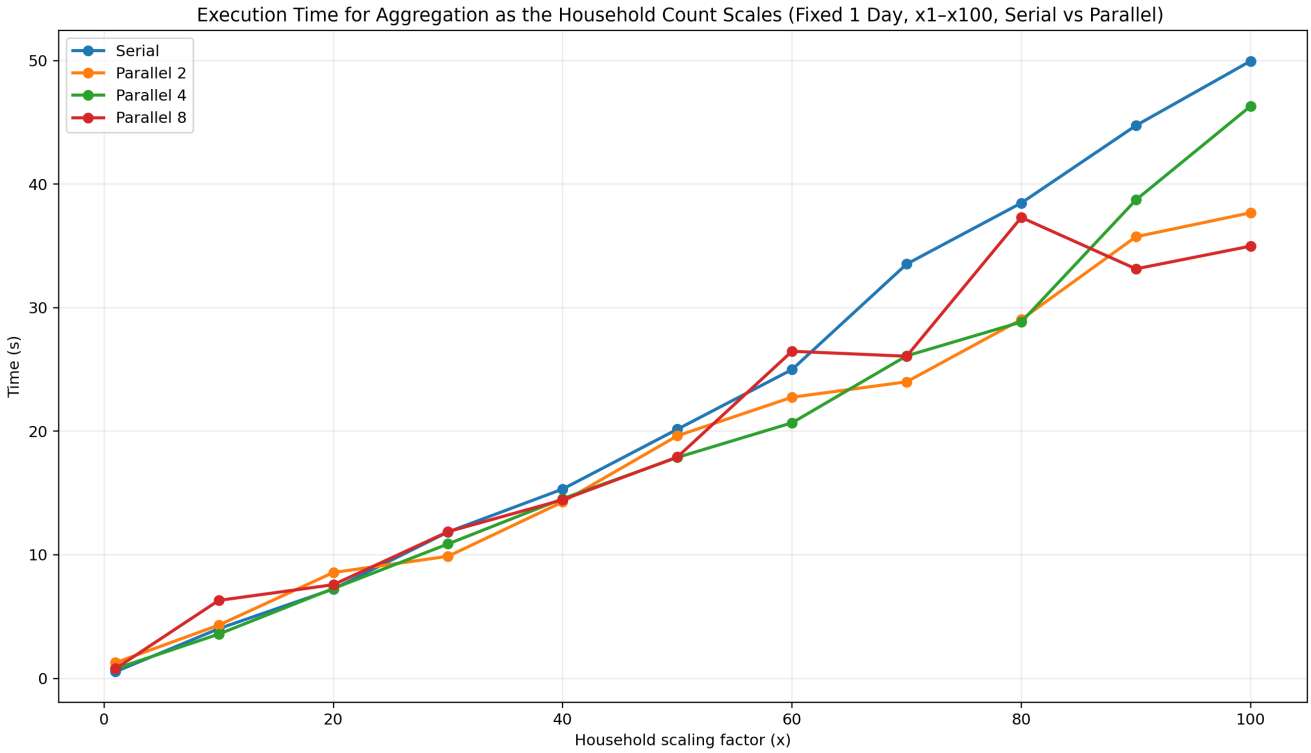


Figure 4: (Key scaling result) Runtime vs. household scaling factor for a fixed 1-day batch (5k households is $\times 1$ up to $\times 100$). This highlights the scale at which parallelism overtakes serial execution.

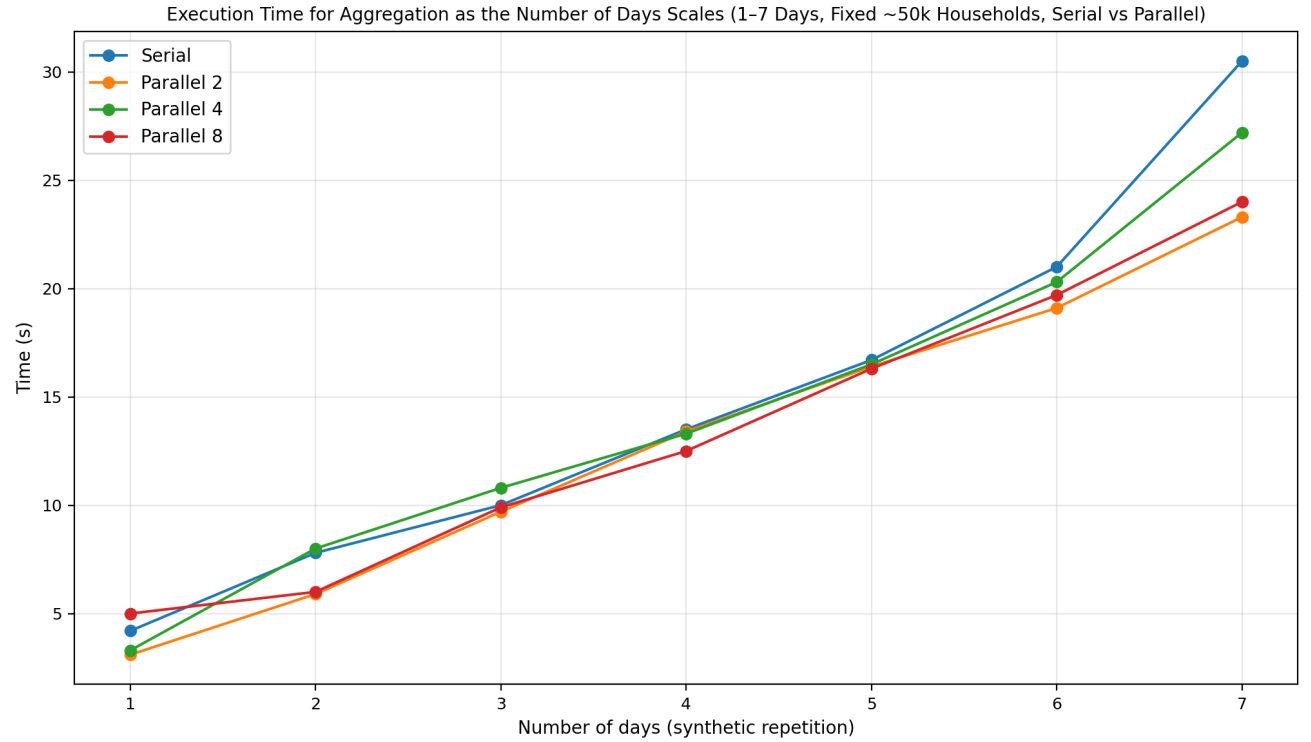


Figure 5: (Key scaling result) Runtime vs. number of days for a fixed $\sim$50,000-household batch. As the batch window grows, parallel execution increasingly outperforms serial due to better amortization of overhead.